

TIE TIY Swing Scanner

Full Master Blueprint

Detailed project document covering the full system thinking, structural reasoning, operating model, specifications, workflow, policy, implementation sequence, and repository plan.

Project Identity	Low-frequency, high-selectivity, execution-ready swing scanner
Market	NSE F&O stocks
Holding Window	2–7 trading days
Output Style	Trade cards with entry, stop, exit, rationale, lifecycle
Primary Design Bias	Precision over coverage
Scope	Full blueprint excluding Claude Code prompt pack

1. Executive Summary

The TIE TIY Swing Scanner is designed as a disciplined, low-frequency, high-selectivity swing trade qualification engine for NSE F&O; stocks. Its purpose is not to generate many signals, entertainment, or broad market excitement. Its purpose is to produce a very small number of serious 2–7 day positional opportunities, each expressed as a complete trade card with clear entry, invalidation, exit logic, and rationale.

The scanner is intentionally built around restraint. Zero trade days are valid. One candidate is often enough. Two candidates is usually the upper bound for a normal day. The scanner's edge is expected to come less from raw setup detection and more from coherent rejection, context awareness, clean trade geometry, and trader usability.

This document consolidates the full project blueprint: foundational reasoning, scanner identity, architecture, gate logic, setup families, execution families, trade card design, workflow, UI, system policy, implementation blueprint, implementation task plan, and repository structure.

Core Mission

Identify a very small number of high-quality, execution-ready positional swing opportunities in NSE F&O; stocks, with meaningful move potential over roughly 2–7 trading days.

Core Constraint

The scanner must prefer restraint over activity. If nothing serious qualifies, it must say no trade explicitly and cleanly.

Core Output

A final trade card, not a vague signal. Every accepted candidate must specify what it is, why it qualified, how it activates, where it is wrong, how to exit, and when it becomes stale.

2. Foundational Thinking

2.1 What problem is actually being solved?

The project is not solving the problem of how to discover many bullish stocks. It is solving the problem of how to identify fewer, stronger, more executable swing trades with meaningful move potential and defined trade geometry.

- The trader does not want random names.
- The trader does not want vague bullish labels.
- The trader does not want a list of charts requiring fresh interpretation every evening.
- The trader wants very few names with clear trade plans and worthwhile move potential.

2.2 Why low frequency is correct

Low frequency is not a weakness here. It is an explicit design choice aligned to the trader's style. The trader prefers 1–2 good names per week over daily activity. Therefore the scanner should optimize for precision, not coverage; decision quality, not idea volume; execution readiness, not exploration.

This means the scanner must be comfortable producing no trades. A no-trade day is not a failure state. It is a correct output when the market does not offer a serious opportunity.

2.3 The single most important mental shift

The scanner is not a signal generator. It is a trade eligibility system. A stock does not deserve a signal merely because something looks bullish. A stock earns eligibility only after passing a sequence of gates that confirm tradability, context fit, structural identity, quality, and execution geometry.

3. Scanner Identity and Objectives

3.1 Project identity

The TIE TIY Swing Scanner is a low-frequency, high-selectivity, positional swing scanner for NSE F&O; stocks, designed to identify 2–7 day opportunities and express them as execution-ready trade cards.

3.2 What the scanner is

- A trade qualification engine
- A context-aware swing opportunity filter
- An execution-ready trade-card generator
- A disciplined operating system with lifecycle memory

3.3 What the scanner is not

- A signal spam tool
- A generic bullish stock finder
- An intraday execution engine
- A broad research dashboard
- An auto-trading system
- A portfolio optimizer
- A prediction machine

3.4 Success definition

The scanner succeeds if it helps the trader answer, quickly and clearly, whether there is a serious swing trade available and, if yes, exactly how to trade it. It is not judged by signal count, cleverness of indicators, or entertainment value.

4. Structural Architecture

The scanner must behave like a qualification funnel, not a flat formula. Each layer solves a distinct problem, and each later layer assumes earlier validity.

```
Universe
→ Market Context Gate
→ Setup Family Gate
→ Quality Gate
→ Tradeability Gate
→ Final Selection Gate
→ Trade Card Output
→ Next-Session Activation
→ Lifecycle Journal
→ Review
```

4.1 Why this architecture exists

- A stock should not be judged on quality if it is not even eligible.
- A setup should not be judged in isolation from broader market context.
- A technically valid setup should still be rejected if trade geometry is poor.
- A final output should be the result of multiple coherent decisions, not one formula.

4.2 Architectural principle

One module, one decision responsibility. One gate, one primary question. One file should not silently do another file's job.

5. Universe and Market Context

5.1 Universe design

The scanner should only evaluate stocks that are structurally eligible to compete.

- NSE F&O; eligible
- Sufficient daily liquidity
- Enough lookback history
- Operationally tradable
- Not structurally broken or obviously low-quality

5.2 Why the universe is intentionally narrow

A broader universe creates more noise, thinner names, worse fills, more false breakouts, and harder reviewability. Since the scanner's purpose is precision, narrowing the universe improves edge quality.

5.3 Market context states

- TREND_SUPPORTIVE
- NEUTRAL_RANGE
- ADVERSE_UNSTABLE

5.4 Context supremacy principle

A candidate does not proceed just because the stock looks attractive. If the broader market does not support the setup family, the candidate must be rejected.

This preserves the rule: good stock + wrong market = bad trade.

6. Setup Family Specification

The scanner is intentionally limited to three approved setup families. These three cover the core behaviors needed for your style: continuation, expansion, and recovery.

6.1 Setup Family A — Trend Continuation Pullback

Behavioral thesis: a strong stock in an established uptrend temporarily pulls back in a controlled manner, then resumes upward.

- Exploits temporary weakness inside strength
- Requires existing uptrend, controlled pullback, support area, and resumption evidence
- Best suited to trend-supportive contexts
- Naturally compatible with 2–7 day continuation moves

6.2 Setup Family B — Compression Break

Behavioral thesis: a stock contracts into a tight structure, stores energy, and then expands directionally when compression resolves.

- Exploits volatility contraction followed by release
- Requires visible compression, intact larger structure, clear breakout boundary, and expansion room
- Typically fast and time-sensitive
- Well suited to short swing opportunities

6.3 Setup Family C — Reclaim / Failed Breakdown Recovery

Behavioral thesis: a stock attempts to break down, fails, reclaims a meaningful level, and begins upward recovery.

- Exploits failed weakness turning into recovery
- Requires meaningful level, failed downside attempt, real reclaim, and recovery path
- Can offer strong asymmetry with clean invalidation
- Useful in selective neutral or supportive environments

6.4 Family policy

A candidate should be assigned one dominant setup family only. If family identity is ambiguous, the scanner should bias toward rejection rather than forcing a weak classification.

7. Quality and Tradeability

7.1 Quality is a separate decision

A setup family being present is not enough. The instance must still be judged.

- Trend quality
- Relative strength
- Participation quality
- Structural location
- Move potential
- Invalidation efficiency

7.2 Tradeability is not optional

A good chart is not automatically a good trade. Final candidates must have clean geometry: a definable trigger, a structural stop, a coherent exit framework, and an expected holding window consistent with 2–7 days.

7.3 The central distinction

The scanner must preserve the difference between:

good-looking stock
versus
good trade candidate

8. Execution Family Specification

Each setup family requires its own execution behavior. Entry, stop, exit, and timing cannot be fully generic.

8.1 Universal execution rules

- Entry must be proof-based, not blind.
- Stop must invalidate the setup thesis, not just control pain.
- Exit must match expected move behavior.
- Time is part of risk; stale behavior matters.
- Skip is a legitimate execution outcome.

8.2 Trend Continuation Pullback execution

- Activation style: resumption confirmation
- Typical entries: above signal candle high, above pullback high, above micro-resistance
- Typical stops: below signal candle low, below pullback low, below continuation support
- Typical exits: first expansion objective, then trail or time stop

8.3 Compression Break execution

- Activation style: breakout confirmation
- Typical entries: above compression high or breakout candle high
- Typical stops: below breakout candle low or compression low
- Typical exits: measured move, first expansion objective, time stop if breakout stalls

8.4 Reclaim Recovery execution

- Activation style: reclaim acceptance confirmation
- Typical entries: above reclaim candle high or above reclaimed level plus confirmation
- Typical stops: below reclaim candle low or failed-breakdown low
- Typical exits: first recovery target, second recovery target or trail, time stop if recovery stalls

9. Gate Specification

The scanner should be governed by a strict gate sequence. Every gate has a purpose, a pass/fail rule, and a reason field.

Gate	Question Answered	Purpose
Gate 0	Is the data usable?	Protect scanner from broken inputs
Gate 1	Is the stock eligible?	Restrict to tradable universe
Gate 2	Is the market supportive?	Context compatibility
Gate 3	Is a valid setup family present?	Behavioral identity
Gate 4	Is this instance good enough?	Quality filtering
Gate 5	Can this be traded cleanly?	Entry/stop/exit geometry
Gate 6	Does it deserve final inclusion?	Low-frequency restraint
Gate 7	Did it really activate next session?	Execution confirmation

9.1 Core gate philosophy

The scanner does not reward partial goodness. Each gate must be passed in sequence. Ambiguity should usually be treated as failure.

9.2 Failure reason policy

Every fail or skip outcome must carry a reason, such as:

- Rejected: insufficient liquidity
- Rejected: setup family not allowed in current market context
- Rejected: weak relative strength and participation
- Rejected: stop too wide relative to expected move
- Skipped: trigger activated too extended
- Expired: valid candidate never triggered within activation window

10. Trade Card Specification

The trade card is the scanner's final decision object. It is not a raw signal, not a research note, and not a label. It is an execution-ready swing trade brief.

10.1 Trade card sections

- Identity block
- Qualification summary block
- Entry block
- Risk / stop block
- Exit block
- Expected move / time block
- Quality / confidence block
- Lifecycle block

10.2 Identity block

- Symbol
- Scan date
- Setup family
- Market context
- Trade status
- Quality grade

10.3 Execution core

- Entry type and entry level
- Activation rule
- Skip rule
- Stop level and stop type
- Invalidation reason
- Target 1
- Target 2 or trailing rule
- Time exit rule

10.4 Why the card matters

The trade card should allow the trader to answer, in under 10 seconds, what the trade is, why it qualified, how it activates, where it is wrong, and what the exit behavior should be.

11. EOD-to-Activation Workflow

The scanner is built around phase separation. EOD is for discovery and planning. The next session is for confirmation and activation.

11.1 Workflow phases

- Phase 0 — Pre-scan readiness
- Phase 1 — EOD scan and qualification
- Phase 2 — Candidate review and trade card preparation
- Phase 3 — Next-session pre-open review
- Phase 4 — Activation check at or after open
- Phase 5 — Post-activation trade state
- Phase 6 — Non-activation / skip / expiry handling
- Phase 7 — Journal and review capture

11.2 Core operational doctrine

A valid EOD card is a conditional opportunity, not an automatic trade. Activation must still be confirmed next session.

11.3 Candidate lifecycle

```
EOD_CANDIDATE
→ PREOPEN_REVIEWED
→ ACTIVATED / NOT_TRIGGERED / SKIPPED / INVALIDATED / EXPIRED
→ if activated: LIVE
→ CLOSED
```

12. Dashboard and UI Specification

The UI must convert scanner logic into disciplined trader decisions. It should feel like a calm control panel, not a research terminal or signal marketplace.

12.1 Main UI surfaces

- Daily Dashboard
- Candidate List View
- Expanded Trade Card View
- Lifecycle / Review View

12.2 Daily Dashboard

- Date
- Scan status
- Last updated time
- Market context
- Scanner state
- Allowed setup families
- Eligible count / candidate count
- Action banner: candidate day / no-trade day / scanner suppressed

12.3 Candidate list

- Symbol
- Setup family
- Quality
- Entry
- Stop
- First exit
- Status
- Short reason

12.4 Expanded trade card

The expanded card is the main operating surface for a trader. It must show the quick action block first, then rationale, then execution logic, then expected behavior, then lifecycle state.

12.5 No-trade display

No-trade must be visually explicit and psychologically normalized. The interface should clearly say when no eligible trade exists and why.

13. System Policy Specification

System policy is the scanner's constitution. It governs how strict the scanner is allowed to be, when it must say no, how many candidates are too many, how activation is handled, and how changes are controlled.

13.1 Core governing rule

When in doubt, preserve clarity, restraint, and tradeability over signal quantity.

13.2 Major policy domains

- Scanner identity policy
- Universe policy
- Context-first policy
- Setup-family limitation policy
- Precision-over-coverage policy
- Tradeability-first policy
- No-trade-is-normal policy
- Activation-is-required policy
- Skip-is-valid policy
- Time-boxed thesis policy
- Change-management policy
- Reviewability policy

13.3 Important policy outcomes

- A good-looking stock may still be rejected.
- No-trade days are acceptable and desirable.
- Thresholds are not changed casually after recent losses.
- The scanner must not broaden itself just to stay busy.

14. End-to-End Implementation Blueprint

The scanner must be built in the correct order. Not the fastest order, not the most exciting order. The correct order.

```
foundation
→ context
→ structure
→ judgment
→ execution
→ presentation
→ memory
```

14.1 Version boundaries

- v1 — core scanner only
- v1.5 — operational refinement
- v2 — enhancement layer

Do not build v2 before v1 is complete and usable.

14.2 Core v1 promise

- Determine market context
- Scan eligible universe
- Detect only valid setup families
- Reject weak candidates
- Build full trade cards
- Support next-session activation
- Log lifecycle cleanly

15. Module Specification

The scanner should be built as a system of tightly scoped modules. Each module must answer one question only.

Module	Primary Question
data_loader	Do we have usable market data?
universe_filter	Which symbols are allowed to compete?
market_context	What kind of market are we scanning in today?
setup_detector	Does this stock show one allowed setup family?
quality_engine	How good is this setup instance?
trade_planner	How exactly should this candidate be traded?
candidate_selector	Which fully valid candidates deserve final inclusion?
output_builder	How should the trader see the result?
journal_manager	What did the scanner show and what happened next?
activation_checker	Did the candidate really trigger next session?
review_analytics	What has the scanner been doing over time?
config_manager	How are settings loaded and validated?

15.1 Module rule

One module = one responsibility. Modules must not quietly absorb another layer's logic. Output builder must not filter. Runner must not contain business logic. Config files must not become hidden code.

16. Implementation Task Plan

The project should be built in six practical phases, each with visible outputs and hard completion tests.

16.1 Practical phases

- Phase 1 — Project skeleton and schemas
- Phase 2 — Data + universe + context spine
- Phase 3 — Setup detection
- Phase 4 — Quality + trade planning
- Phase 5 — Output + workflow
- Phase 6 — Journal + activation + review

16.2 Weekly framing

- Week 1: skeleton, schemas, placeholders, configs, runner stub
- Week 2: data loading, universe eligibility, market context
- Week 3: setup family detection
- Week 4: quality engine and trade planner
- Week 5: selector, dashboard, no-trade handling
- Week 6: journaling, activation, review analytics

16.3 Ground rules during implementation

- Build v1 only
- One module at a time
- Do not polish UI before logic is usable
- Do not add more setup families
- Every task must produce a visible and testable result

17. Repository File Plan

The repository must reflect architecture. It should feel like an operating system, not a notebook graveyard or script dump.

```
tietiy-swing-scanner/  
  ■■■ README.md  
  ■■■ requirements.txt  
  ■■■ .gitignore  
  ■■■ configs/  
  ■■■ data/  
  ■■■ docs/  
  ■■■ outputs/  
  ■■■ journals/  
  ■■■ tests/  
  ■■■ runner/  
  ■■■ src/  
    ■■■ tietiy_scanner/  
      ■■■ __init__.py  
      ■■■ schemas.py  
      ■■■ constants.py  
      ■■■ utils.py  
      ■■■ data_loader.py  
      ■■■ universe_filter.py  
      ■■■ market_context.py  
      ■■■ setup_detector.py  
      ■■■ quality_engine.py  
      ■■■ trade_planner.py  
      ■■■ candidate_selector.py  
      ■■■ output_builder.py  
      ■■■ journal_manager.py  
      ■■■ activation_checker.py  
      ■■■ review_analytics.py  
      ■■■ config_manager.py
```

17.1 Repo rule

If someone opens the repo, it should be obvious where data logic lives, where policy lives, where outputs go, where lifecycle is stored, and which files are runners versus real business logic.

18. Build Risks and Completion Criteria

18.1 Major build risks

- Building presentation too early while logic is weak
- Adding too many setup variants too early
- Mixing module responsibilities
- Letting threshold changes become emotional
- Treating no-trade days as scanner failure

18.2 v1 completion criteria

- Runs on clean data
- Classifies context consistently
- Detects only named setup families
- Rejects weak candidates
- Produces full trade cards
- Shows no-trade days clearly
- Logs full lifecycle

18.3 Final test

The system is only truly complete when, at EOD, it can decide whether today matters, identify only serious candidates, generate complete cards, support next-session activation, and later reconstruct the full lifecycle of what it showed and what happened.

19. Final Conclusion

This project is not a scanner in the shallow sense. It is a full trade qualification system. Its edge is expected to come from structural discipline: a narrow universe, explicit market context, a small number of named setup families, a strict quality layer, strong tradeability requirements, calm UI, explicit no-trade handling, and complete lifecycle memory.

The central idea of the whole blueprint is simple:

Do not ask: "Is this stock bullish?"

Ask: "Has this candidate earned trade eligibility as a serious 2-7 day swing trade?"

If the build stays faithful to that question, the scanner will remain coherent, tradable, and reviewable. If it drifts toward quantity, constant activity, or loose definitions, it will become noisy and lose its edge.

This blueprint is the master reference for the project.

End of document.